

Smart Home Automation System

Midway Technical Report — Circuit Analysis, ROS 2 & GUI Architecture

Engineering Team: CrocoMarine

1 System Overview & Architecture

This technical report presents the Smart Home automation, environmental monitoring, and safety platform engineered by team **CrocoMarine**. The system integrates low-level microcontroller hardware simulated in Proteus with a multi-node ROS 2 software suite and PyQt user interface.

Key System Capabilities

- **Environmental Monitoring:** Real-time acquisition of temperature, relative humidity, combustible gas levels, ambient light intensity, and indoor occupancy/motion.
- **Automated Safety Actuation:** Automated PWM ventilation fan speed control, position-controlled servo window venting, emergency buzzer alarms, and transistor-driven ambient lighting arrays.
- **Dual Control Interfaces:** On-site 4×4 keypad with I²C LCD, complemented by a workstation ROS 2 Qt dashboard with historical CSV data logging.

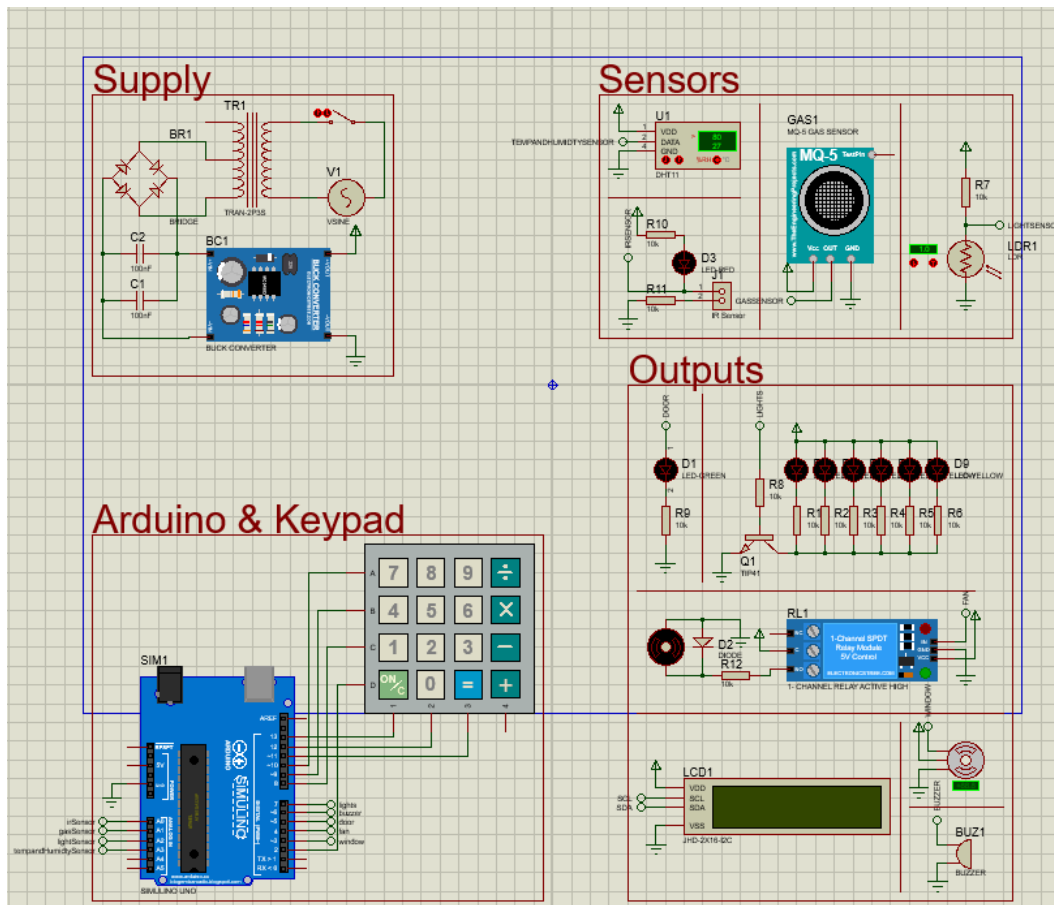


Figure 1: Complete Smart Home System Schematic in Proteus.

As illustrated in Figure 1, the hardware architecture is partitioned into four functional modules:

1. **Supply Block:** Steps down and rectifies utility AC to a DC bus before regulation to +5 V.
2. **Sensors Block:** Transduces physical quantities into calibrated digital and analog signals.
3. **Arduino & Keypad Block:** Central processing unit handling signal processing, local PIN security, and UART communication.
4. **Outputs Block:** Actuation systems for environmental control, door locking mechanisms, and alarms.

2 Subsystem Circuit Analysis

2.1 Power Supply Subsystem (Supply)

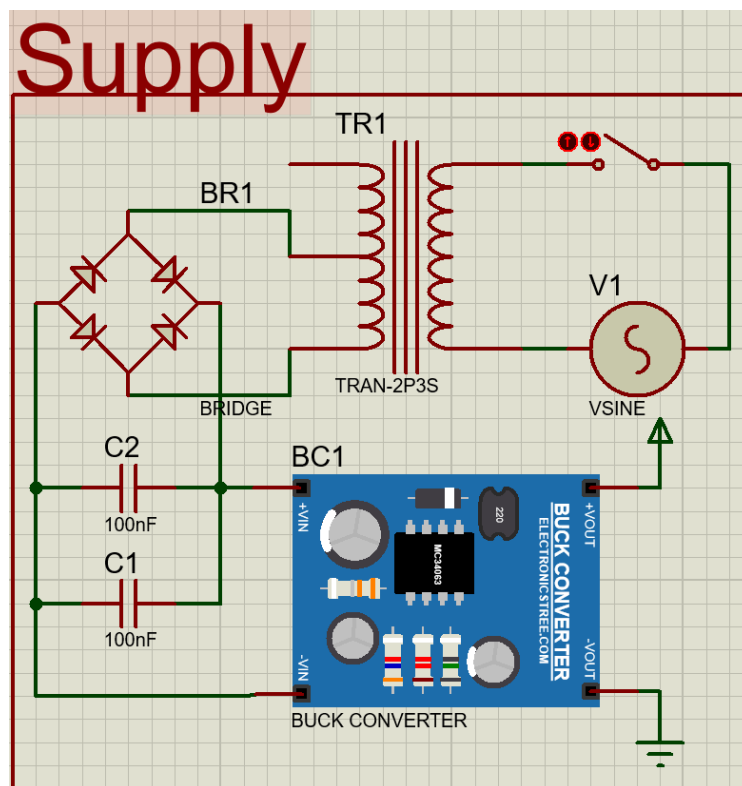


Figure 2: Power Supply Subsystem Schematic (Supply).

The power conversion stage (Figure 2) reduces high-voltage utility AC to an intermediate rectified DC voltage, which is then regulated down to a stable +5 V DC power rail for digital microcontrollers, sensors, and switching drivers.

2.1.1 Component Selection & Circuit Operation

- **AC Source (V1 / VSINE):** Simulates 220 V AC utility mains input.
- **Step-Down Transformer (TR1):** A 220 V to 24 V AC **center-tapped step-down transformer**. It steps down high-voltage AC while providing galvanic isolation between high-voltage mains and low-voltage control circuits.
- **Bridge Rectifier (BR1):** Configured with the center-tapped transformer output for full-wave rectification, yielding a rectified DC voltage of 18.09 V DC under load across the primary smoothing bus.

- **Filtering Stage** ($C_1 = 50\ \mu\text{F}$, $C_2 = 100\ \text{nF}$):
 - $C_1 = 50\ \mu\text{F}$ acts as the primary bulk electrolytic filter capacitor to smooth low-frequency 100 Hz/120 Hz voltage ripple from the rectifier.
 - $C_2 = 100\ \text{nF}$ ceramic capacitor filters out high-frequency switching transients and noise spikes.
- **DC-DC Switching Regulator (BC1 – MC34063)**: Buck converter stepping down the 18.09 V DC input rail to a regulated +5 V output rail.
 - **Efficiency Rationale**: Switching regulators operate at high efficiency ($> 85\%$), avoiding the large thermal power loss of linear regulators ($P_{\text{loss}} = (V_{\text{in}} - V_{\text{out}}) \times I_{\text{load}}$) when powering relays, servos, and LED arrays.

2.2 Sensor Suite Subsystem (Sensors)

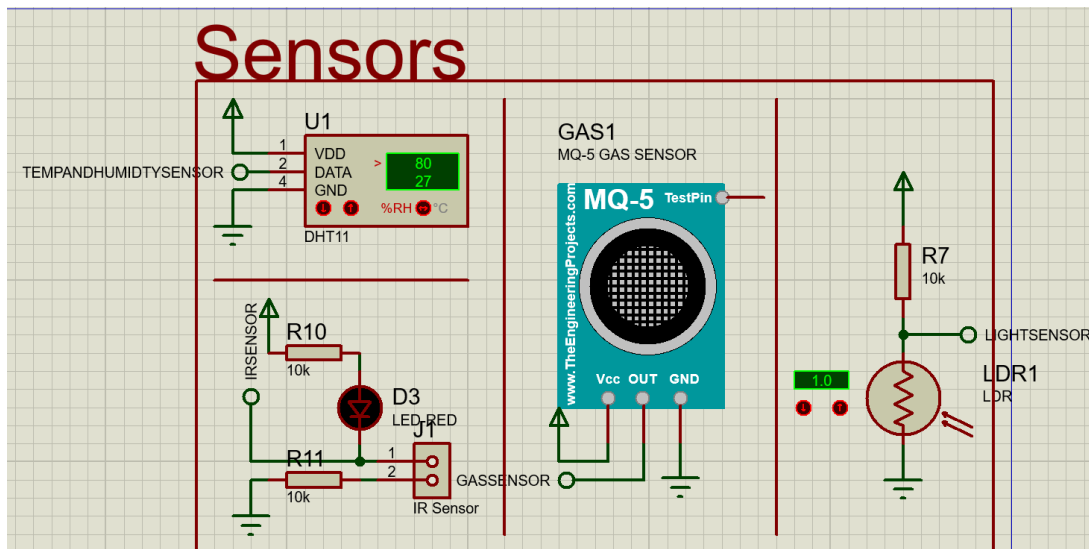


Figure 3: Sensor Suite Subsystem Overview (Sensors).

The sensor module (Figure 3) converts physical ambient conditions into readable electronic signals.

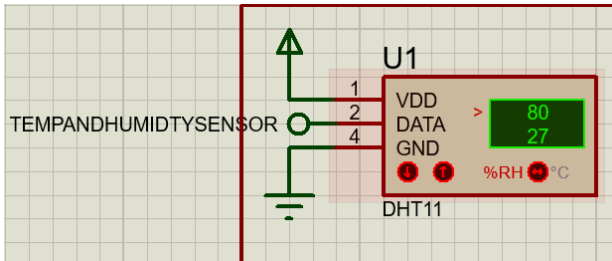


Figure 4: DHT11 Digital Sensor.

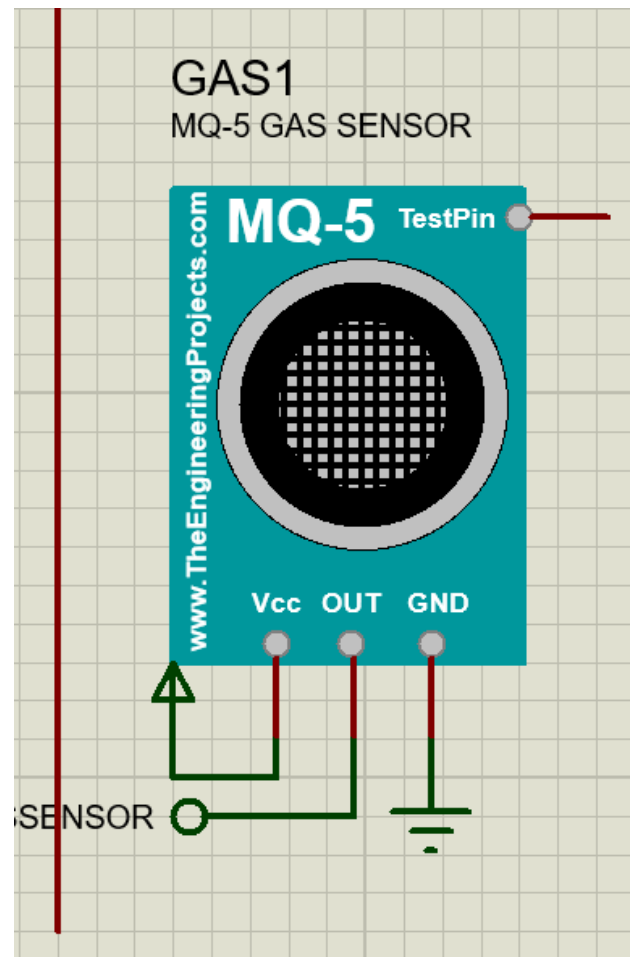


Figure 5: MQ-5 Gas Sensor.

2.2.1 DHT11 Temperature & Humidity Sensor (U1)

- **Operation:** Reads temperature (0°C to 50°C) and relative humidity (20% to 90% RH).
- **Connections:** DATA (Pin 2) connected to net TEMPANDHUMIDITYSENSOR (Arduino Analog pin A3). Digital single-bus transmission eliminates ADC conversion drift.

2.2.2 MQ-5 Combustible Gas Sensor (GAS1)

- **Operation:** Detects hydrocarbon gas concentration (LPG, methane, natural gas).
- **Connections:** OUT pin connected to net GASENSOR (Arduino Analog pin A1). Analog voltage increases proportionally with gas concentration.

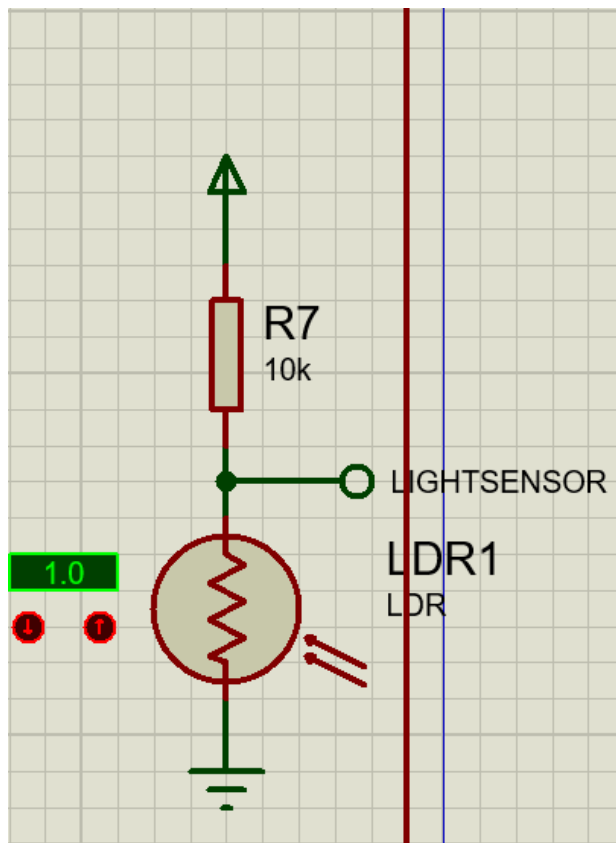


Figure 6: LDR Voltage Divider.

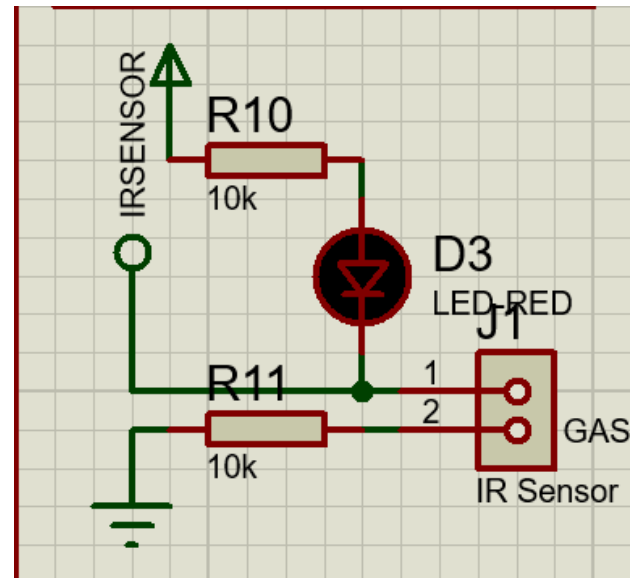


Figure 7: IR Motion Sensor Interface.

2.2.3 Light Sensor Circuit (LDR1 + R_7)

- **Voltage Divider Formulation:** LDR1 forms a divider with fixed resistor $R_7 = 10\text{ k}\Omega$:

$$V_{\text{LIGHTSENSOR}} = V_{CC} \times \frac{R_{\text{LDR}}}{R_7 + R_{\text{LDR}}} \quad (1)$$

- **Design Selection:** With nominal ambient room resistance around $10\text{ k}\Omega$, selecting $R_7 = 10\text{ k}\Omega$ centers nominal dark/light voltage transition near 2.5 V , optimizing ADC measurement range.

2.2.4 IR Motion Sensor Interface (J_1 , R_{10} , R_{11} , D3)

- **Resistor Values & Function:** $R_{10} = 10\text{ k}\Omega$ limits indicator LED D3 current. $R_{11} = 10\text{ k}\Omega$ acts as a pull-down resistor on net IRSENSOR, preventing floating inputs when no motion is present.

2.3 Microcontroller & User Interface Subsystem (Arduino & Keypad)

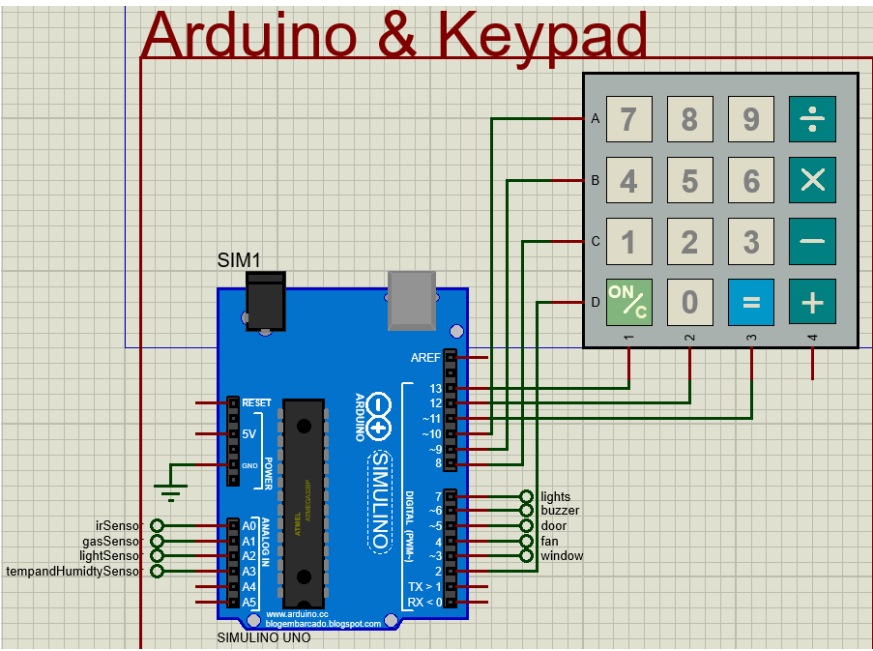


Figure 8: Arduino Uno and 4 × 4 Keypad Interface Block.

The central processing unit is an ATmega328P-based Arduino Uno (SIMULINO UNO) shown in Figure 8.

Table 1: Arduino Pin Assignments and Hardware Mapping

Arduino Pin	Signal Label	Hardware Target / Peripheral Function
A0	irSensor	IR Motion Module Input (J1)
A1	gasSensor	MQ-5 Gas Sensor Analog Output (GAS1)
A2	lightSensor	LDR Voltage Divider Node (LDR1)
A3	tempandHumiditySensor	DHT11 Digital Serial Data (U1)
Pins 8 – 13	Keypad Bus	4 × 4 Matrix Keypad Row/Column Scanning Bus
Pin 7	lights	NPN Transistor Switch Driver → Room LED Array
Pin 6	buzzer	Acoustic Piezo Alarm Unit (BUZ1)
Pin 5	door	Electronic Door Lock Indicator (D1)
Pin 4	fan	SPDT Relay Module (RL1) → Ventilation Motor
Pin 3	window	Servo Motor Signal (Venting Window Actuator)

2.4 Actuation & Output Subsystem (Outputs)

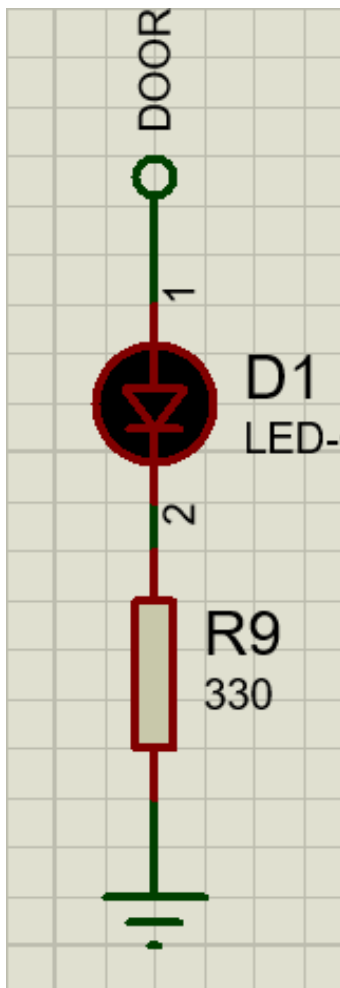


Figure 9: Door Lock LED Circuit.

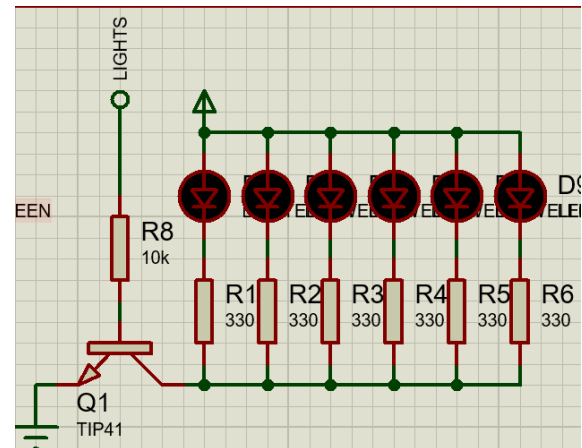


Figure 10: Lighting Array Transistor Driver.

2.4.1 Electronic Door Lock Indicator (DOOR)

- Green LED D1 in series with $R_9 = 330\ \Omega$ driven by Pin 5.
- Limiting resistor $R_9 = 330\ \Omega$ sets output current to $I_D = \frac{5V - 2V}{330\ \Omega} \approx 9.1\text{ mA}$, providing visual feedback upon successful PIN validation or GUI override.

2.4.2 High-Power Smart Lighting Driver (LIGHTS)

- Drives six parallel LEDs (D4–D9) via NPN Power Transistor Q1 (TIP41) and Base resistor $R_8 = 10\text{ k}\Omega$.
- **Low-Side Switching:** Prevents pin overload on the Arduino. Base resistor $R_8 = 10\text{ k}\Omega$ drives Q1 into full saturation, safely switching the total load current ($\approx 60\text{ mA}$). Individual $330\ \Omega$ resistors (R_1 – R_6) equalize branch currents.

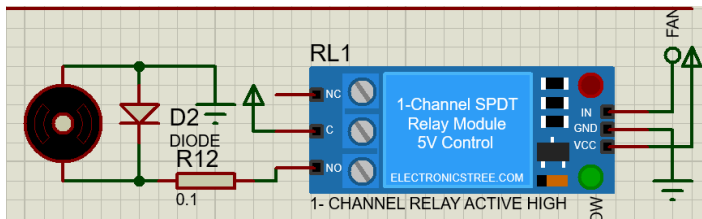
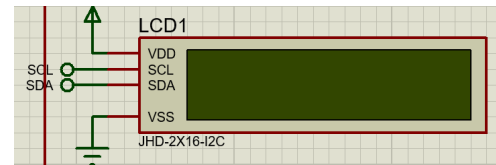


Figure 11: Ventilation Fan Relay Driver.

Figure 12: I²C LCD Module.

2.4.3 Ventilation Fan Relay Control (FAN)

- 1-Channel SPDT 5 V Relay Module (RL1) isolates low-voltage control signals from motor noise.
- **Protection & Monitoring:** Flyback Diode D2 suppresses inductive back-EMF spikes during switching. Shunt resistor $R_{12} = 0.1\ \Omega$ enables current sensing.

2.4.4 User Information Display (LCD1)

- PCF8574 I²C expansion adapter reduces control connections to two lines: Serial Data (SDA) and Serial Clock (SCL).

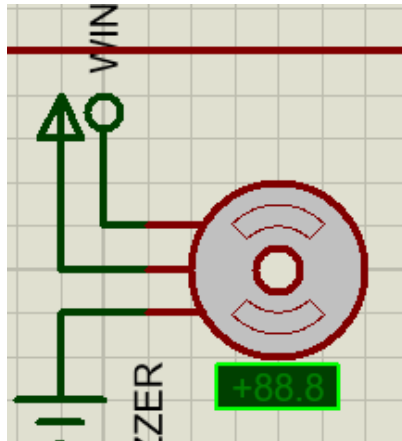


Figure 13: Window Vent Servo Actuator.

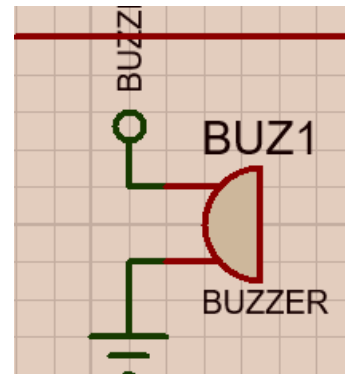


Figure 14: Audible Alarm Buzzer.

2.4.5 Automated Window Servo Actuator (WINDOW)

- PWM-controlled Servo Motor driven by Digital Pin 3. Enables precise angular vent positioning (0° closed to 90° open) during hazard events or manual GUI commands.

2.4.6 Audible Security & Safety Alarm (BUZZER)

- Piezoelectric Buzzer BUZ1 driven by Pin 6 for acoustic alert patterns during emergency gas conditions or invalid keypad access attempts.

3 ROS 2 System Architecture & Software Design

The system software is structured around **ROS 2 (Robot Operating System 2)**, divided into three dedicated packages:

1. **serial_receiver Node:** Handles low-level UART communication with the Arduino hardware, parsing sensor telemetry streams into ROS 2 topics and publishing serial command strings to actuators.

2. **control_unit Node:** Evaluates environmental safety rules, threshold logic, and automated response actions across system nodes.
3. **gui_control Node:** PyQt-based workstation application providing real-time data visualization and explicit manual overrides.

3.1 ROS 2 Dashboard & Historical Telemetry Interface

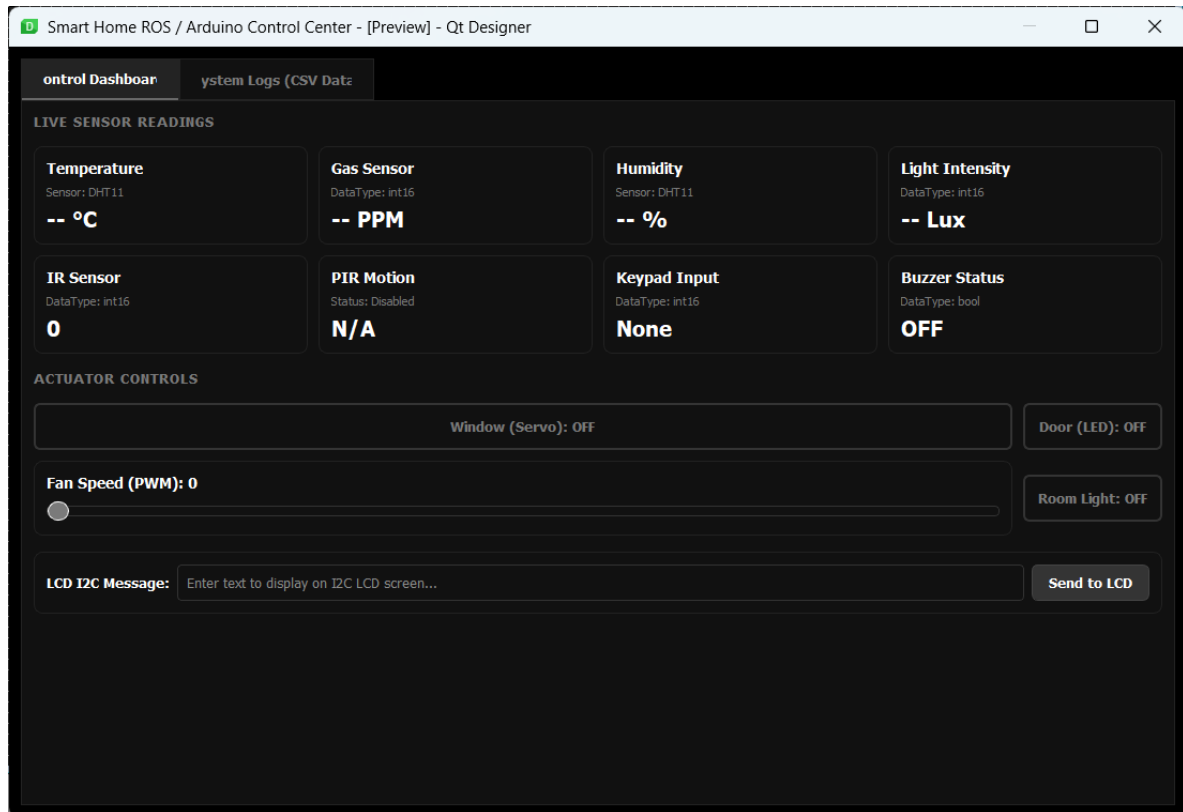


Figure 15: ROS 2 / Qt Control Center - Live Telemetry Dashboard and Manual Overrides.

The primary dashboard interface (Figure 15) includes:

- **Real-Time Telemetry Gauges:** Displays live values for Temperature (°C), Gas (PPM), Humidity (% RH), Light (Lux), IR Motion, Keypad status, and Buzzer state.
- **Manual Override Controls:** Actuator controls for Window position, PWM Fan slider, Door Lock toggle, Room Lighting array, and custom message text entry for the I²C LCD display.

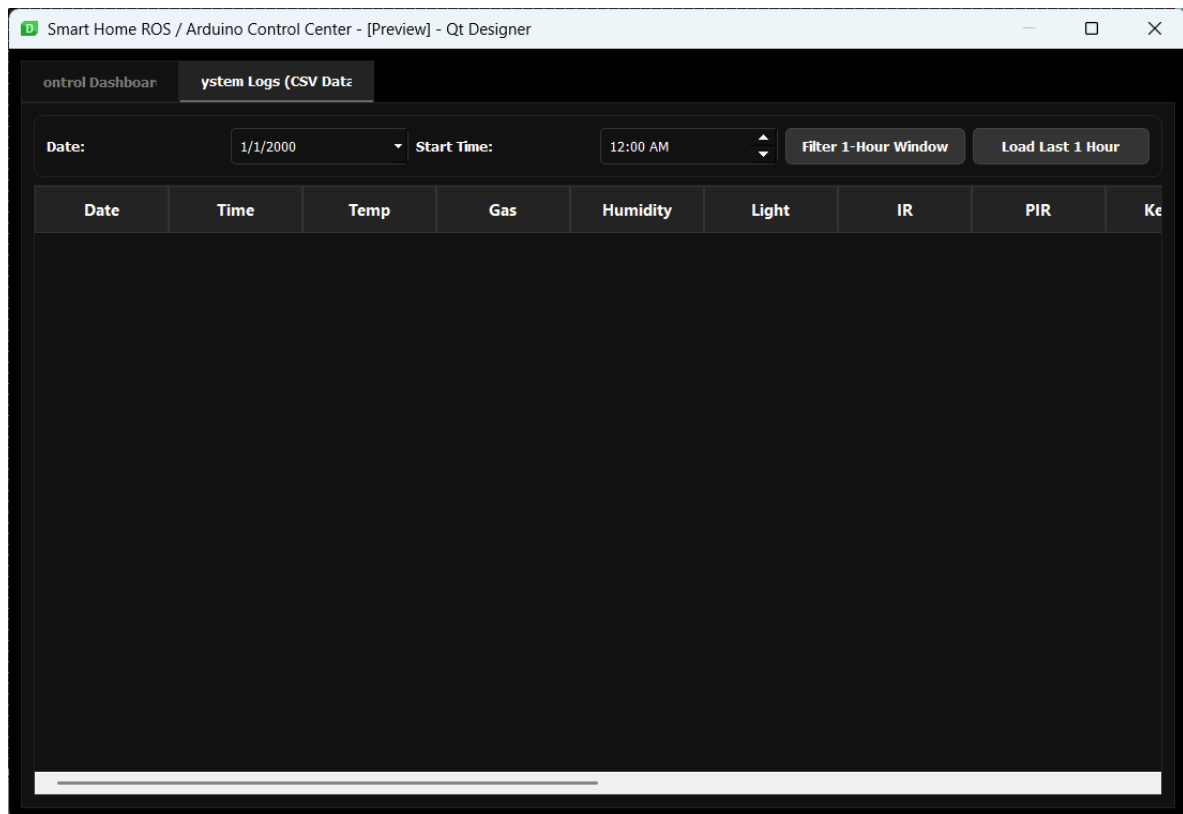


Figure 16: ROS 2 / Qt Control Center - Historical System Data Viewer (CSV Logs).

The historical logging interface (Figure 16) aggregates sensor logs and published event frames into structured CSV files, allowing historical filtering by Date, Time, and 1-Hour sample intervals.

4 System Control Logic

Automated Emergency Safety Routines

- **Combustible Gas Detection:** If **GAS1** exceeds safety thresholds → Activate acoustic alert (**BUZZER**), engage ventilation fan (**FAN Relay RL1**), drive window servo (**WINDOW**) to 90° open position, and push alert notifications to the LCD and ROS 2 dashboard.
- **Climate Regulation:** Temperature over threshold → Activates **FAN PWM**. High ambient humidity → Drives **WINDOW** servo to open ventilation position.
- **Smart Lighting Routine:** Low ambient light level (**LDR1**) OR motion detected (**IR**) → Triggers transistor switch **Q1**, illuminating room LED array **D4–D9**.
- **Security Entry Logic:** Correct PIN entry → Drives **D00R** indicator HIGH (**D1**). Incorrect PIN attempts → Triggers alert patterns via **BUZZER**.

5 Project Resources & Repositories

- **Working Hardware Video Demonstration (Google Drive):**
https://drive.google.com/drive/folders/1r1NdAYJq_-MU7d_LZ1VdzBB-Kjb_msCU?usp=sharing
- **Complete Source Code & ROS 2 Package Repository (GitHub):**
<https://github.com/WEZXY/MidWayCroco.git>